

B.Sc. (Data Science)

DS101T : Problem Solving and Python Programming

Unit 1: Introduction to Problem Solving

Mrs. Reshma Masurekar

**Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18**

Unit1: File Handling

- 1.1 Introduction to Files
- 1.2 Types of Files
- 1.3 Opening and Closing a Text File
- 1.4 Writing to a Text File
- 1.5 Reading from a Text File
- 1.6 Setting Offsets in a File
- 1.7 Creating and Traversing a Text File

Need of file

- Till now Python programs are written to accept the input, manipulate it and display the output. But that output is available only during execution of the program and input is to be entered through the keyboard. This is because the variables used in a program have a lifetime that lasts till the time the program is under execution. What if we want to store the data that were input as well as the generated output permanently so that we can reuse it later? Usually, organizations would want to **permanently store information** about employees, inventory, sales, etc. **to avoid repetitive tasks of entering the same data. Hence, data are stored permanently on secondary storage devices for reusability.** We store Python programs written in script mode with a .py extension. Each program is stored on the secondary device as a file. Likewise, the data entered, and the output can be stored permanently into a file.

Introduction to Files

- A file is a named location on a **secondary storage media** where data are **permanently** stored for later access.
- Python has several functions for file handling to perform operations like Creating, Reading, Updating, Deleting files.

Types of Files

- Computers store every file as a collection of 0s and 1s i.e., in binary form. Therefore, every file is basically just a series of bytes stored one after the other.
- There are mainly **two types of data files** -
 1. **Text file**
 2. **Binary file**

Text file

- A **text file** consists of human readable characters, which can be opened by any text editor.
- A **text file** can be understood as a sequence of characters consisting of alphabets, numbers and other special symbols.
- Example: Files with extensions like .txt, .py, .csv, etc. are some examples of text files.

Binary files

- **Binary files** are made up of non-human readable characters and symbols, which require specific programs to access its contents.

Sr. No.	Text File	Binary file
1	Text files contains only text data such as letters, digits, and punctuation marks.	Binary files contains non-textual data, such as images, audio files, and executables.
2	Opened in read mode ("r"), write mode ("w") or append mode("a")	Opened in read mode ("rb"), write mode ("wb"), or append mode ("ab")
3	Generally smaller in size compared to binary files.	Generally larger in size compared to text files.
4	There is a delimiter EOL (End of Line)	There is no delimiter.
5	Text files are easy to understand.	Binary files are difficult to understand.

Working of open () function

FileObject= open (filename, access_mode)

- We use open () function in Python to open a file in read or write mode. open () will return a file object.
- To return a file object we use open () function along with two arguments, that accepts file name and the mode, whether to read or write.

access_mode	Description
x	Create - Creates the specified file, returns an error if the file exists.
r	Read - Default value. Opens a file for reading, error if the file does not exist.
w	Write - Opens a file for writing, creates the file if it does not exist.
a	Append - Opens a file for appending, creates the file if it does not exist.
r+	To read and write data into the file. This mode does not override the existing data, but you can modify the data starting from the beginning of the file.
w+	To write and read data. It overwrites the previous file if one exists, it will truncate the file to zero length or create a file if it does not exist.
a+	o append and read data from the file. It won't override existing data.

Create a New File

- **Create a New File:**
- To create a new file in Python, `open()` method is used, with one of the following parameters:
- **"x" - Create** - will create a file, returns an error if the file exists.
- **"a" - Append** - will create a file if the specified file does not exists.
- **"w" - Write** - will create a file if the specified file does not exists.

Opening a file

- `open()` function opens the file if possible and returns the corresponding file object.

- **Syntax :**

FileObject = open("filename", "mode")

where,

- **file** – A path-like object which represents the file path
- **mode (Optional)** – The mode is an optional parameter. It's a string that specifies the mode in which you want to open the file.

Opening a file

```
f = open("myfile.txt", "x")
```

Result: a new empty file is created!

```
f = open("myfile.txt", "w")
```

Result: Create a new file if it does not exist!

```
f = open("demofile.txt")
```

```
f = open("demofile.txt", "rt")
```

Both the statements are same.

Result: Because "r" for read, and "t" for text are the default values, you do not need to specify them.

Note: Make sure the file exists, or else you will get an error.

Closing the file

- In order to close a file, first open the file.
- In python, in-built method called `close ( )` is used to close the file which is opened.
- The file will remain open until you close the file using the `close()` function.
- Whenever a file is opened, it is important to close it, especially, with write method. Because if the `close()` function is not called after the write method then whatever data written to a file will not be saved into the file.
- **Syntax:**
`FileObject.close()`

Opening a file using with clause

- In Python, file can also be opened using **with clause**.
- **Syntax:**
with open (file_name, access_mode) as file_object:
- The advantage of using with clause is that any file that is opened using this clause is closed automatically, once the control comes outside the with clause.
- In case, the user forgets to close the file explicitly or if an exception occurs, the file is closed automatically.

Example:

```
with open("new.txt","r") as f:  
    print(f.read())
```

Reading from a Text File

- **Steps to Read Text File in Python**
- In Python, to read a text file, follow the below steps:
 - Step 1:** Open the file for reading using the `open()` method and pass a file path to the function if required.
 - Step 2:** The next step is to read the file, and this can be achieved using several built-in methods such as `read()`, `readline()`, `readlines()`.
 - Step 3:** Once the read operation is performed, the text file must be closed using the `close()` function.

Working of read () mode

- There are three ways to read the contents of a file:
 - `read()`
 - `readline([n])`
 - `readlines()`

read()

- **read()** function returns the read bytes in the form of a string. This method is useful when you have a small file, and you want to read the specified bytes or entire file and store it into a string variable.
- **Syntax:**

FileObject.read()

```
file1 = open("data.txt", "r")  
print(file1.read(7))
```

It will return a first 'n' number of characters(first 7 characters)

To create a new empty file

```
file1=open("data.txt","x")
```

To read data from a file

```
file1=open("abc.txt", "r")  
print(file1.read())  
file1.close()
```

To read data from a file

```
file1 = open("data.txt","r")  
print(file1.read())  
file1.close()  
file1 = open("data.txt","w")  
file1.write("Hello World")  
file1.close()
```

readline([n])

- readline([n]) - This method **reads one complete line from a text file** and returns in the form of a string, where each line terminates with a newline (\n) character.
- It can also be used **to read a specified number (n) of bytes of data from a file but maximum up to the newline character (\n).**
- Syntax:

FileObject.readline()

- **Example:**
- **file1.readline(7):** Read and return a first 7 characters of first line of the file
- **file1.readline():** Read and return a first line of the file

```
file1 = open("data.txt", "r")  
print(file1.readline(7))  
file1.close()
```

It will return a first 'n' number of characters from first line.(first 7 characters)

```
file1 = open("data.txt", "r")  
print(file1.readline())  
file1.close()
```

It will return a first line

```
file1=open("data.txt", "r")  
print(file1.readline())  
print(file1.readline())  
file1.close()
```

It will return a first two lines

readlines()

- `readlines()` function reads all the lines from the text file and returns each line as a string element in a list.
- This function can be used for small files, as it reads the whole file content to the memory, then split it into separate lines. We can iterate over the list and strip the newline “\n” character using `strip()` function.
- Syntax:

`FileObject.readlines()`

```
file1 = open("data.txt", "r")  
print(file1.readlines())
```

A **loop** is used to **read the entire file line by line** using the `readline()`. This process is known as looping/ iterating over a file object. It returns an empty string when EOF is reached.

```
file1=open("data.txt", "r")  
for line in file1:  
    print(line.strip())
```

```
file1=open("data.txt", "r")  
f1=file1.readlines()  
for line in f1:  
    print(line.strip())
```

write()

- will overwrite any existing content.
- Syntax:

FileObject.write(inputString)

```
f = open("new.txt", "w")  
print("Name of the file: ", f.name)  
f.write("This is the new line")  
print("Check the file to see the reflected changes")  
f.close()
```

writelines()

- Allows to write a list of string to the file in a single call. used to write multiple strings to a file. We need to pass an iterable object like lists, tuple, etc. containing strings to the writelines() method. Unlike write(), the writelines() method does not return the number of characters written in the file
- **Syntax:**
`FileObject.writelines(list)`
- The list of texts or byte objects that will be inserted.

```
f = open("demofile3.txt", "a")
f.writelines(["See you soon!", "Over and out."])
f.close()
#open and read the file after the appending:
f = open("demofile3.txt", "r")
print(f.read())
```

Setting Offsets in a File

- To access the contents of file randomly:
 - `tell()`
 - `seek()`

tell()

- **tell()** function returns the integer giving current position of file object in a file stream.
- The integer returned specifies the number of bytes from the beginning of file till the current position of file object.
- By file object i.e. a cursor. And it's cursor, who decides from where data has to be read or written in a file.
- **Syntax:**

FileObject.tell()

Function returns the current file position.

seek()

- seek() function in Python is used to **move the file cursor to the specified location**.
- When we read a file, the cursor starts at the beginning, but we can move it to a specific position by passing an arbitrary integer (based on the length of the content in the file) to the seek() function.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
P	Y	T	H	O	N	P	R	O	G	R	A	M	M	I	N	G

seek(3)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
P	Y	T	H	O	N	P	R	O	G	R	A	M	M	I	N	G

- **Syntax:**

FileObject.seek(offset, [from_what])

Where,

***FileObject** is file pointer*

Parameters:

Offset: *Reuired Parameter; Number of positions to move forward.*

from_what: *Optional Parameter; It defines point of reference to start from which place.(0/1/2); The default is 0.*

0 – Default. Sets the point of reference at the **beginning of the file**.
Equivalent to os.SEEK_SET.

1 – Sets the point of reference at the **current position of the file**.
Equivalent to os.SEEK_CUR.

2 – Sets the point of reference at the **end of the file**.
Equivalent to os.SEEK_END.

Returns: *Return the new absolute position.*

The reference point is selected by the **from_what** argument.

Example:

f.seek(5) : Keeps the file pointer at reads the file content from 6th position onwards to till EOF.

```
f=open("data.txt","r")  
print(f.tell())  
print(f.seek(4))  
print(f.read())
```

Output:

```
0  
4  
Science  
Computer Science Department  
Dr D Y Patil ACS College  
Pimpri Pune 18
```

```
f=open("data.txt","rt")  
print(f.seek(10,1))  
print(f.read())
```

Output:

```
10  
b'ce\r\nComputer Science  
Department\r\nDr D Y Patil ACS  
College \r\nPimpri Pune 18'
```

```
f=open("data.txt","rb")  
print(f.seek(10,1))  
print(f.read())
```

Output:

```
io.UnsupportedOperation: can't do  
nonzero cur-relative seeks
```

```
f=open("data.txt","rb")  
print(f.seek(-10,2))  
print(f.read())
```

Output:

```
74  
b'ri Pune 18'
```

```
fname=input("enter filename:")  
f=open(fname,"r")  
print("file Current Position: ", f.tell())  
print("file contents are:\n", f.read())  
print("file Position after reading the file: ", f.tell())  
print("file contents are: \n", f.read())  
print("Set the file pointer at begining..")  
print(f.seek(10))  
print(f.read())
```

Output:

```
enter filename:data.txt  
file Current Position: 0  
file contents are:  
Data Science  
Computer Science Department  
Dr D Y Patil ACS College  
Pimpri Pune 18  
file Position after reading the file: 84  
file contents are:
```

```
Set the file pointer at begining..  
10  
ce  
Computer Science Department  
Dr D Y Patil ACS College  
Pimpri Pune 18
```

```
fname=input("enter filename:")  
f=open(fname,"r")  
print("file Current Position: ", f.tell())  
print("file contents are:\n", f.read())  
print("file Position after reading the file: ", f.tell())  
print("file contents are: \n", f.read())  
print("Set the file pointer at begining..")  
print(f.seek(10))  
print(f.read())
```

Output:

```
enter filename:data.txt  
file Current Position: 0  
file contents are:  
Data Science  
Computer Science Department  
Dr D Y Patil ACS College  
Pimpri Pune 18  
file Position after reading the file: 84  
file contents are:
```

```
Set the file pointer at begining..  
10  
ce  
Computer Science Department  
Dr D Y Patil ACS College  
Pimpri Pune 18
```

Creating and Traversing a text file

- To create a text file, the `open ()` method is used which contains filename and the filemode. If the file already exists with the same name, the `open()` function will behave differently depending on the mode (write or append) used. If it is in write mode (`w`), then all the existing contents of file will be lost, and an empty file will be created with the same name. But, if the file is created in append mode (`a`), then the new data will be written after the existing data. In both cases, if the file does not exist, then a new empty file will be created.

To create a text file and write data in it

```
f=open("demo.txt","w+")
```

```
while True:
```

```
    data= input("Enter data to save in the text file: ")
```

```
    f.write(data)
```

```
    ans=input("Do you wish to enter more data?(y/n): ")
```

```
    if ans=='n': break
```

```
f.close()
```

Python program to demonstrate traversing file using **while loop**

```
f=open("first.txt","r")
```

```
str = f.readline()
```

```
while str:
```

```
    print(str)
```

```
    str=f.readline()
```

```
f.close()
```

Write a Python Program to perform the following:

Write Operation:

The `write_to_file()` function opens a file in 'write' mode ('w'), which creates the file if it doesn't exist, or truncates it (empties it) if it does exist.

The content is written into the file.

Read Operation:

The `read_from_file()` function opens the file in 'read' mode ('r') and reads the content.

If the file does not exist, a `FileNotFoundError` is caught and a message is returned

Function to write to a file

```
def write_to_file(filename, str):  
    with open(filename, "w") as file:  
        file.write(str)  
    print(f"Content written to {filename}")
```

Function to read from a file

```
def read_from_file(filename):  
    try:  
        with open(filename, "r") as file:  
            content = file.read()  
        return content  
    except FileNotFoundError:  
        return f"The file {filename} does not exist."
```

File name

```
filename = "example.txt"
```

Writing content to the file

```
str = "Hello, this is a sample content for the file. \n Let's perform file operations in Python."  
write_to_file(filename, str)
```

Reading content from the file

```
file_content = read_from_file(filename)  
print("\nContent read from file:")  
print(str)
```